Universidad de Antioquia
Departamento de Electrónica y Telecomunicaciones
2598521 - Informatica II

Fiesta mundial con UdeAWorldCup

Informe de Análisis y Diseño Técnico

Rigoberto Berrio Berrio
Estudiante de Ingeniería Electrónica
rigoberto.berrio1@udea.edu.co

Felipe Herrera Lopera
Estudiante de Ingeniería Electrónica
felipe.herreral@udea.edu.co

27 de abril de 2026

1. Análisis del problema y consideraciones de la solución

1.1. Contexto del desafío

El desafío consiste en diseñar e implementar un sistema capaz de simular el desarrollo completo de una Copa Mundial de fútbol, utilizando el lenguaje C++ dentro del entorno Qt Creator.

Este tipo de problema no solo implica modelar entidades simples, sino representar un sistema complejo donde interactúan múltiples componentes: equipos, jugadores, partidos y estadísticas. Cada uno de estos elementos posee comportamiento propio y además afecta el estado global del sistema.

A diferencia de ejercicios básicos, este proyecto introduce dificultades reales de ingeniería de software, tales como:

- Coordinación entre múltiples clases interdependientes.
- Manejo correcto de memoria dinámica en estructuras complejas.
- Separación clara entre lógica, datos y persistencia.
- Simulación coherente de eventos dinámicos (partidos).

En este sentido, el problema se asemeja a un sistema real, donde no basta con que el programa funcione, sino que debe ser organizado, escalable y mantenible.

1.2. Restricciones y decisiones de diseño

Durante el desarrollo no solo se siguieron las restricciones del enunciado, sino que también se tomaron decisiones que impactaron directamente la calidad del sistema.

Algunas de las más relevantes fueron:

- **Uso de Programación Orientada a Objetos (POO):** Se decidió modelar cada elemento del sistema como una clase independiente. Esto permitió representar el problema de forma más natural, similar a cómo ocurre en el mundo real.
- **Separación en múltiples archivos (.h y .cpp):** Aunque inicialmente parecía más complejo, esta decisión facilitó mucho la organización del código a medida que el proyecto crecía.
- **Uso de memoria dinámica:** Fue una de las partes más desafiantes. Se utilizó para permitir estructuras flexibles (como listas de jugadores), aunque implicó tener cuidado con fugas de memoria y punteros inválidos.
- **Evitar dependencia excesiva de STL:** Esto obligó a implementar estructuras propias, lo cual ayudó a comprender mejor cómo funcionan internamente los arreglos dinámicos.
- **Uso de archivos CSV solo para inicialización:** Se decidió que los datos externos solo se usaran al inicio, evitando complejidad innecesaria durante la simulación.

Üna restricción técnica determinante fue la prohibición del uso de structs, contenedores STL y funciones ANSI C como strcpy, strlen o sprintf. Esto obligó a implementar funciones utilitarias propias dentro de cada clase para la gestión de cadenas de caracteres mediante char* con memoria dinámica, así como una plantilla genérica ArregloDinamico(T) que reemplaza completamente a std::vector. Adicionalmente, la simulación de partidos exigió modelar un proceso estocástico basado en la distribución de Poisson, parametrizado con datos históricos reales de cada selección.

1.3. Estrategia general de solución

La solución se diseñó siguiendo una arquitectura modular basada en responsabilidades claras.

Primero, se identificaron las entidades principales del dominio:

- Equipos
- Jugadores

Luego, se definieron las clases encargadas de la lógica del sistema:

- Partido
- Grupo
- Torneo

Adicionalmente, se incluyeron módulos auxiliares para:

- Manejo de archivos
- Estadísticas
- Estructuras dinámicas

2. Esquema de tareas y arquitectura del sistema

2.1. Estructura modular

El sistema se organiza en varios módulos principales:

Módulo	Responsabilidad
entidades	Representación de Equipo, Jugador y relaciones
simulacion	Lógica de partidos, grupos y torneo
estadisticas	Cálculo y almacenamiento de métricas
persistencia	Lectura de archivos CSV
estructuras	Implementación de arreglos dinámicos
utilidades	Funciones auxiliares del sistema
main.cpp	Punto de entrada y ejecución del sistema

2.2. Diagrama de dependencias

Las dependencias entre módulos siguen una jerarquía estricta de abajo hacia arriba. La capa de estructuras (ArregloDinamico(T)) no depende de ningún otro módulo, ya que es la base del sistema. La capa de estadísticas (EstHistEquipo, EstHistJugador, EstPartidoEquipo, EstPartidoJugador) depende únicamente de las estructuras. La capa de entidades (Equipo, Jugador) depende de estadísticas y estructuras. La capa de simulación (Partido, Grupo, Torneo) depende de entidades y estadísticas. La capa de persistencia (GestorArchivos) depende de entidades. Esta arquitectura garantiza que los cambios en capas superiores no afecten a las inferiores.

El flujo de dependencias del sistema refleja una jerarquía lógica:

- **main.cpp** actúa como punto de entrada
- El módulo de simulación coordina el comportamiento global

- Las entidades representan los datos base
- Las estructuras y persistencia soportan el funcionamiento interno

2.3. Secuencia de ejecución del sistema

1. Se cargan los datos iniciales desde archivos CSV.
2. Se crean dinámicamente los objetos de equipos y jugadores.
3. Se organizan los equipos en grupos.
4. Se simulan los partidos dentro de cada grupo.
5. Se calculan estadísticas y posiciones.
6. Se avanza a fases eliminatorias.
7. Se ejecutan eliminatorias hasta obtener un campeón.

3. Algoritmos implementados

3.1. Fórmula de simulación de goles

La simulación de goles por partido se fundamenta en un modelo paramétrico simplificado mediante la siguiente ecuación para el parámetro λ del equipo A:

$$\lambda_A = \mu \cdot \left(\frac{GF_A}{\mu}\right)^{\alpha} \cdot \left(\frac{GC_B}{\mu}\right)^{\beta} \tag{1}$$

donde $\mu = 1{,}35$, $\alpha = 0{,}6$ y $\beta = 0{,}4$. GF_A representa el promedio histórico de goles a favor del equipo A y GC_B el promedio de goles en contra del equipo B. El parámetro λ se interpreta como los goles esperados por el equipo A en ese partido específico.

Para distribuir los goles entre jugadores, se itera sobre los once convocados y se aplica una probabilidad del 4 % de anotar por jugador, hasta alcanzar el total de goles esperados redondeado. Esta distribución es independiente entre jugadores y se evalúa con el generador de números aleatorios estándar de C++.

3.2. Algoritmo de programación de fechas

La asignación de fechas a los 72 partidos de la fase de grupos implementa un algoritmo voraz (*greedy*). Para cada partido, el algoritmo busca secuencialmente el primer día disponible dentro de una ventana de 19 días que satisfaga dos condiciones simultáneas:

1. El día no supera el límite de 4 partidos globales permitidos.
2. Ninguno de los dos equipos involucrados ha disputado un partido en los últimos 3 días.

La complejidad temporal del algoritmo es $\mathcal{O}(P \times D)$, donde P es el número de partidos y D el número de días disponibles. Este enfoque *greedy* no garantiza la solución óptima en términos de distribución uniforme, pero sí garantiza la satisfacción de todas las restricciones del enunciado.

3.3. Conformación de grupos con restricciones de confederación

El sorteo de grupos sigue un proceso de cuatro etapas. Primero se forman cuatro bombos ordenados por ranking FIFA, reservando el primer lugar del bombo 1 para Estados Unidos como país sede, asignado directamente al Grupo A. Luego se realiza el sorteo aleatorio extrayendo un equipo de cada bombo para cada grupo.

La validación de restricciones de confederación se aplica por intento: para cada bombo y cada grupo, se selecciona aleatoriamente un equipo candidato y se verifica que su confederación no exceda el cupo permitido (máximo 1 por confederación, máximo 2 para UEFA). Si el equipo no es compatible, se reintenta con otro candidato. En caso de agotar todos los intentos sin éxito, se realiza una asignación forzada para garantizar que el sorteo termine.

3.4. Criterios de desempate en la tabla de clasificación

La tabla de cada grupo se ordena aplicando los siguientes criterios en orden de prioridad:

1. Mayor puntaje acumulado (3 por victoria, 1 por empate, 0 por derrota).
2. Mayor diferencia de goles en partidos del grupo ($GF - GC$).
3. Mayor cantidad de goles a favor en el grupo.
4. Sorteo aleatorio como último recurso.

Para la transición a R16, los 12 terceros de grupo se clasifican por los mismos criterios y solo los 8 mejores avanzan.

3.5. Formato de archivos de persistencia

El sistema utiliza dos archivos CSV separados. El primero, `selecciones_clasificadas_mundial.csv`, es provisto por el enunciado y contiene los datos históricos de los 48 equipos separados por punto y coma. El *parser* propio implementado en `GestorArchivos` lee el archivo carácter a carácter mediante `ifstream::getline`, detectando el separador `;` y el carácter de fin de línea en formato Windows (`\r\n`).

El segundo archivo, `jugadores.csv`, es generado por el sistema con el siguiente formato por fila:

```
PAIS;CAMISETA;NOMBRE;APELLIDO;PJ;GOLES;MINUTOS;ASIST;AMARILLAS;ROJAS;FALTAS
```

Se eligió un archivo único para todos los jugadores en lugar de un archivo por equipo por eficiencia: una sola apertura de archivo permite leer o escribir los datos de los 1248 jugadores (48×26), evitando el *overhead* de 48 operaciones de apertura y cierre. El campo `PAIS` actúa como clave de búsqueda para asociar cada fila con su equipo correspondiente.

4. Problemas de desarrollo afrontados

Durante el desarrollo del proyecto se presentaron múltiples dificultades, especialmente relacionadas con la complejidad del sistema y el manejo de memoria.

Uno de los principales problemas fue el uso de memoria dinámica, ya que en varias ocasiones se generaron fugas de memoria o accesos inválidos. Esto obligó a revisar cuidadosamente el uso de punteros y la liberación de recursos mediante los operadores `delete` y `delete[]`.

Un problema recurrente fue la violación del calificador `const` en métodos que retornaban referencias a colecciones internas. La solución fue implementar versiones duales de los *getters* — una mutable y una `const`

— para todos los métodos que retornan referencias a `ArregloDinamico<T>`. Este patrón debió aplicarse sistemáticamente en las clases `Grupo`, `EstPartidoEquipo`, `Equipo` y `Jugador`.

Adicionalmente, el uso de la palabra reservada `final` como nombre de variable generó errores de compilación en C++11, requiriendo el renombramiento de variables locales en el método `generarEstadisticas()`. De igual forma, la función `strcpy` generó advertencias de seguridad del compilador, lo que motivó su reemplazo por implementaciones propias basadas en iteración de caracteres, en cumplimiento estricto de la prohibición de funciones ANSI C.

También se presentaron inconvenientes al sincronizar los datos leídos desde archivos con los objetos creados en memoria, lo que generaba inconsistencias en algunos casos. Otro problema importante fue la organización inicial del proyecto, donde el código estaba poco modularizado, lo que dificultaba realizar cambios sin afectar otras partes del sistema. Esto llevó a una refactorización completa hacia una arquitectura modular con separación de responsabilidades clara.

5. Evolución de la solución y consideraciones de implementación

5.1. Evolución del diseño

Inicialmente, el sistema tenía una estructura monolítica. Con el tiempo, se refactorizó hacia una arquitectura modular, separando responsabilidades y mejorando la mantenibilidad del código.

La evolución más significativa del diseño fue la transición de una clase `Fecha` independiente — que inicialmente encapsulaba día, mes y año con sobrecarga de operadores aritméticos (`+`, `-`, `<`, `==`) — hacia su integración directa como tres atributos enteros dentro de la clase `Partido`. Esta decisión redujo el número de clases de 11 a 10, respondiendo al criterio de diseño orientado a la comunicación efectiva del modelo más que a la granularidad máxima de abstracción.

Igualmente, la clase `EntradaTabla` fue inicialmente concebida como `struct`, lo cual violaba las restricciones del enunciado. Posteriormente se eliminó como entidad independiente, integrando la lógica de la tabla de clasificación directamente en `Grupo` mediante arreglos paralelos de tamaño fijo 4, aprovechando que el número de equipos por grupo es constante y conocido en tiempo de compilación.

5.2. Consideraciones prácticas de implementación

- **Memoria dinámica:** todas las clases que contienen punteros implementan destructor explícito, constructor de copia profunda (*deep copy*) y operador de asignación, siguiendo la regla de los tres de C++.
- **Separación lógica y datos:** la clase `Partido` encapsula toda la lógica de simulación, mientras que las clases de estadísticas solo almacenan datos, sin ningún comportamiento propio de simulación.
- **Modularidad:** cada módulo puede modificarse independientemente sin afectar al resto del sistema, gracias a la jerarquía estricta de dependencias entre capas.
- **Eficiencia por referencia:** los parámetros de métodos que reciben objetos grandes se pasan por referencia (`const T&`) en lugar de por valor, evitando copias innecesarias de estructuras complejas como `Equipo` o `Grupo`.

6. Diagrama de clases

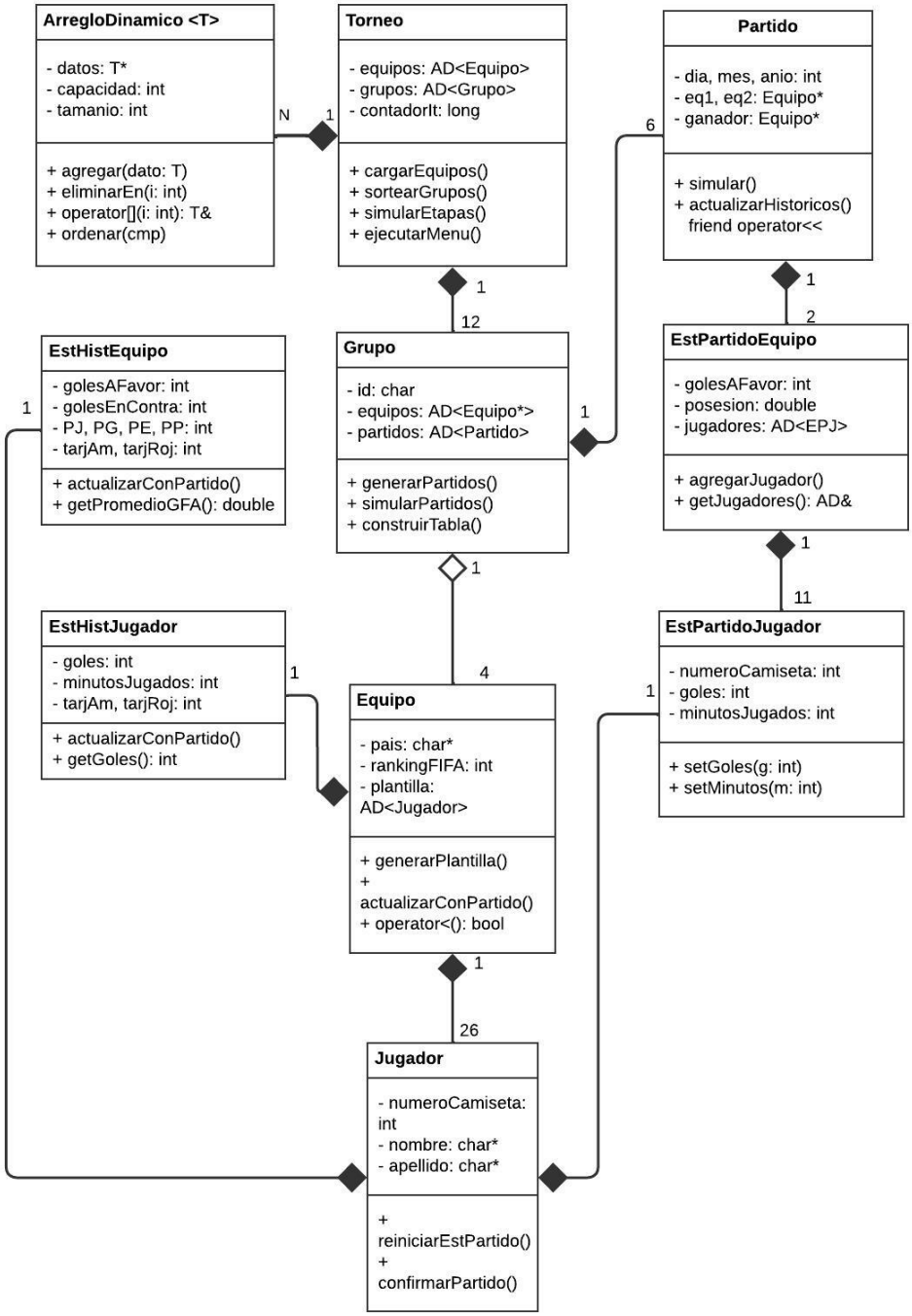

Figura 1: Diagrama de Clases de la Solución UdeAWorldCup.

7. Conclusiones

El sistema UdeAWorldCup demuestra que es posible construir una simulación estocástica de complejidad real respetando restricciones de diseño. La implementación de `ArregloDinamico<T>` como única estructura de datos dinámica evidencia que la prohibición de la STL, no limita el sistema, por el contrario profundiza la comprensión de los mecanismos internos de los contenedores.

La separación entre estadísticas históricas y estadísticas de partido resultó ser la decisión de diseño más importante del proyecto, ya que el método `confirmarPartido()`, que transfiere los resultados del partido actual al historial acumulado, solo es posible gracias a esta separación. Si no se hubiesen separado no habría permitido reinicializar el estado temporal sin comprometer el historial permanente.

El uso de plantillas de clase mediante el mecanismo `template` de C++ permitió implementar una única estructura de datos genérica reutilizable para todos los tipos del sistema, eliminando duplicación de código.